

XGBoost in Action: Detecting Fraud in Transaction Data

BOUJIDA Nezar

12/07/2025

Abstract

Payment-card fraud, while rare, inflicts disproportionate financial and reputational damage. We train an XGBoost classifier on 587 299 anonymized transactions (0.24% fraudulent). Evaluated on a strictly held-out December test set, our model attains an Average Precision of 0.672, achieving 88% fraud recall and only a 7% false-positive rate at a 0.5 decision threshold. These results underscore XGBoost's effectiveness for scalable, real-time fraud detection under extreme class imbalance.

Contents

1	Dataset Overview	2
2	Introduction	3
3	Problem Statement	3
4	Key Results	3
5	Exploratory Data Analysis: Feature Discrimination	4
5.1	Monthly Transaction Trends And Fraud Rates	4
5.2	Fraud Rates by Card-Issuing Country	4
5.3	Payment Channel and Card-Reading Method	4
5.4	Inter-Transaction Time Analysis	5
5.5	Inter-Transaction Time Analysis	6
5.6	Top Currencies: Transaction Share vs. Fraud Rate	6
6	Feature Preparation and Transformation	7
6.1	Chronological Data Splitting	7
6.2	Currency Fraud-Risk Encoding	7
6.3	Feature Selection and Encoding	7
7	Model Setup	7
7.1	Hyperparameter Grid Search	7
8	Boosting-Round Performance Evaluation	8
8.1	Test-Set Performance	8
8.2	Feature Importance	8
9	appendix	9

1 Dataset Overview

The study uses a card-transaction dataset comprising 587 299 records and 13 attributes, with a fraud incidence of approximately 0.24% (1 393 frauds vs. 585 906 legitimate transactions). Features fall into the following groups:

- **Identifiers**

- `transaction_id` (unique, hashed)
- `card_id` (hashed)

- **Categorical**

- Issuer country (`country_of_registration`)
- Processing country (`processing_country`)
- Payment channel (`payment_method`)
- Card-reading method (`card_payment_method`)
- Currency (`payment_currency`)

- **Temporal**

- Date of transaction (`time_of_transaction`)

- **Quantitative**

- Transaction amount (`transaction_amount`)
- Time delta since prior transaction (`difference_transaction_time`)
- Time delta since second-prior transaction (`difference_transaction_time2`)

2 Introduction

In recent years, payment card fraud has emerged as a major challenge for banks, merchants, and consumers alike. The rapid growth of digital transactions driven by online shopping, contactless payments, and mobile wallets offers convenience but also opens the door to new and increasingly sophisticated attacks.

Machine learning techniques have become a key tool in the fight against fraud, enabling automated detection of unusual patterns in large volumes of transaction data.

Among these methods, gradient-boosted decision trees, particularly the XGBoost implementation are widely adopted for their balance of predictive power, computational efficiency, and ease of integration.

3 Problem Statement

Payment card fraud poses a critical threat to financial institutions and consumers, costing the industry billions annually while eroding customer trust.

In our anonymized dataset of 587 299 transactions, only 0.24% are fraudulent, creating an extreme class imbalance that renders naïve classifiers ineffective. Moreover, every false alarm blocking or reviewing a legitimate transaction carries its own cost in customer inconvenience and operational overhead.

Consequently, the core challenge is to develop a model that reliably detects the rare fraud cases (maximizing fraud recall) while maintaining a low false-positive rate, all under the constraints of high throughput and real-time decisioning.

4 Key Results

On the held-out December test set (53 948 transactions, 217 frauds), our final XGBoost model achieves an **Average Precision (AP) of 0.672**¹, demonstrating strong ranking ability even under extreme class imbalance.

At the default 0.5 decision threshold, the model correctly flags 191 of the 217 fraud cases yielding a *fraud recall* of 88% while preserving 49 941 of 53 731 genuine transactions (*non-fraud recall* of 93%, corresponding to a 7% false-positive rate).

Figure 1 displays the December test-set Precision–Recall curve. We calculate the random-ranking baseline as the fraud prevalence (0.40%), and our model achieves an Average Precision of 0.672 far above this baseline, underscoring its ability to maintain high precision even as recall climbs.

Figure 2 presents the confusion matrix at the 0.5 threshold: the top-left cell and bottom-right cell reflect our conservative yet effective flagging catching most frauds while minimizing false alarms.

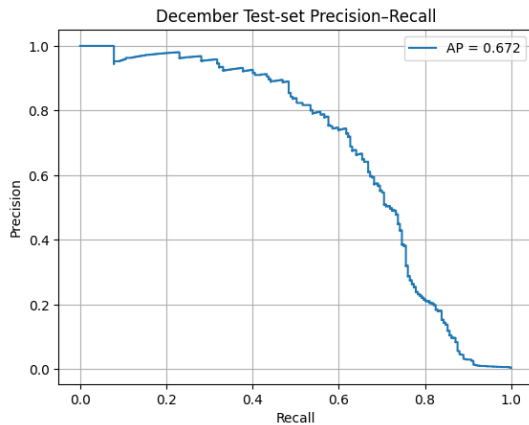


Figure 1: December test-set Precision–Recall curve (AP = 0.672).

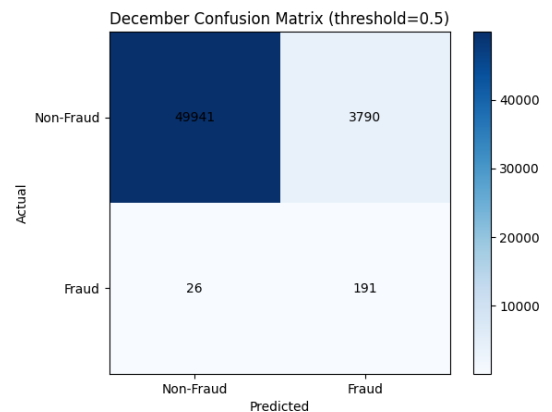


Figure 2: December confusion matrix at threshold = 0.5.

¹See Appendix 9 for a detailed interpretation of Average Precision.

5 Exploratory Data Analysis: Feature Discrimination

In this section we assess each attribute’s capacity to differentiate fraudulent from legitimate transactions.

5.1 Monthly Transaction Trends And Fraud Rates

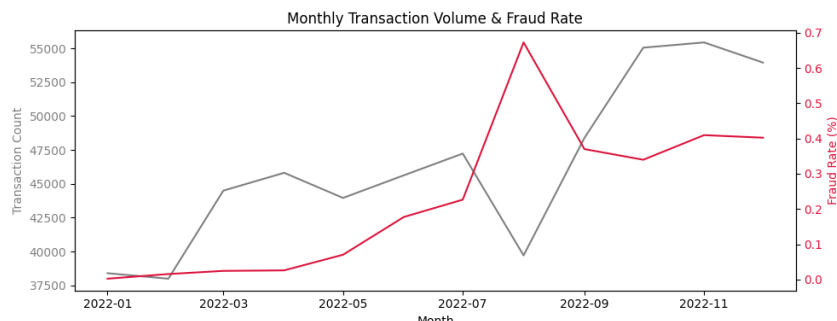


Figure 3: Monthly Transaction Volume & Fraud Rate.

Transaction volume grew steadily from 38 000 in January to a November peak of 55 000 before a slight December dip, while fraud rate rose from near 0 % to about 0.4 % by year-end. Notably, August saw a volume trough (39 000) paired with a fraud spike (0.68 %), revealing a mid-summer anomaly.

5.2 Fraud Rates by Card-Issuing Country

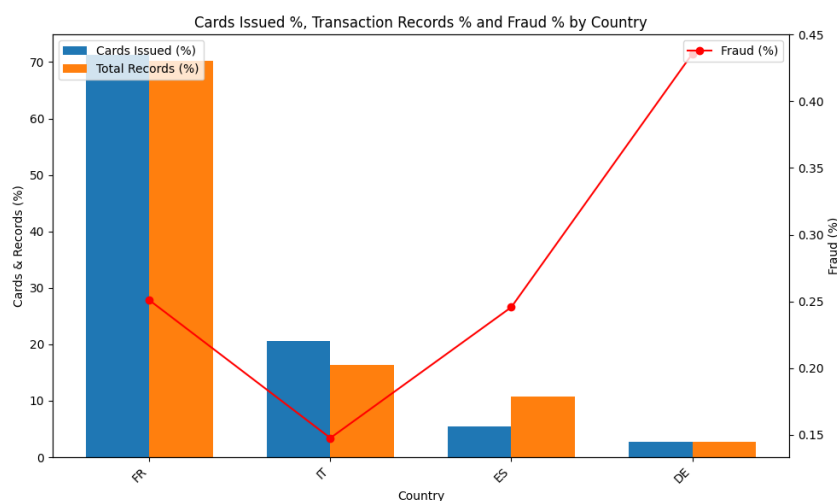


Figure 4: Cards issued, transaction share, and fraud rate by issuing country.

Leading the pack, France issues 71.3% of all cards and generates 69% of transactions, with a fraud rate of 0.25%, a level that underlines effective risk controls at scale.

Next, Italy issues 20.6% of cards and records 17% of transactions, yet contains fraud to just 0.13%, the lowest among the four markets.

Spain: Though responsible for only 5.4% of issued cards, Spain handles 10% of transaction volume and posts a 0.24% fraud rate nearly matching France despite its smaller footprint.

Finally, Germany: With 2.7% of cards and 4% of transactions, Germany endures the highest fraud incidence at 0.42%.

5.3 Payment Channel and Card-Reading Method

Card-Reading Method (Figure 5b). Nearly all card payments about 95 % are processed via VAD, and this method also posts the highest fraud rate at roughly 0.25 %. Transactions falling into the “Other” category make up only around 3% and see a very low fraud incidence of 0.05 %. By contrast, EMV chip reads account for just 1–2 % of transactions and have recorded zero fraud, showing the strong security

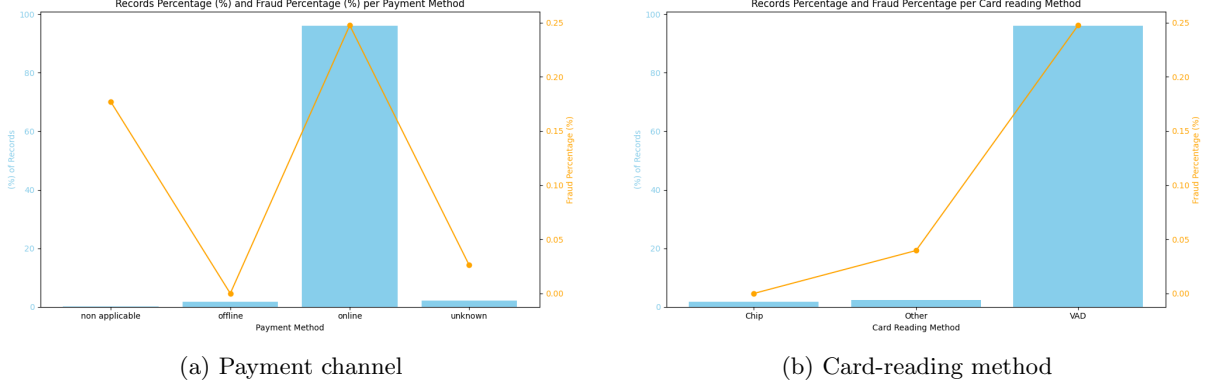


Figure 5: Transaction share (bars) and fraud rate (line) for the two main categorical features.

profile of chip-based authorization.

A chi-square test of independence confirms these differences are highly significant ($\chi^2 = 46.02$, $p \approx 1 \times 10^{-10}$). Under the null hypothesis of equal fraud rates, we would have expected approximately 23 frauds on Chip reads (observed = 0) and about 30 on “Other” methods (observed = 5), whereas VAD would account for about 1273 frauds (observed = 1321). This statistical validation underscores that card-reading method is a powerful predictor: VAD carries elevated risk, “Other” methods remain low-risk, and Chip reads are effectively immune in our sample.

Payment Channel (Figure 5a). The blue bars show each payment method’s share of total transactions, while the orange line traces their fraud rates. Online payments dominate (95%) and incur the highest fraud rate (0.25%). The non-applicable category, though only about 1% of volume posts the second-highest fraud rate (0.18%), signalling that this small bucket is surprisingly risky. By contrast, both offline and unknown methods (each 2% of transactions) register virtually no fraud.

To ensure these differences aren’t just random noise, we performed a chi-square test of independence:

- Null hypothesis (H_0): Fraud incidence is the same across all payment methods.
- Test result: $\chi^2 = 47.00$ with $df = 3$ and $p \approx 3.5 \times 10^{-10}$.

Such a small p-value lets us reject H_0 , confirming that payment method and fraud incidence are dependent, with online and non-applicable channels showing significantly higher risk than offline or unknown.

5.4 Inter-Transaction Time Analysis

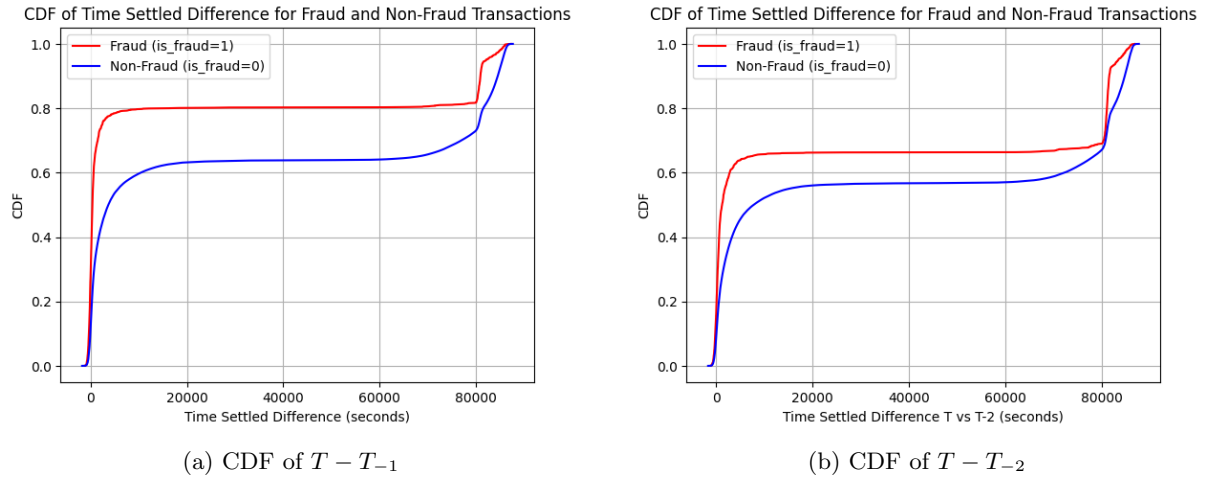


Figure 6: Empirical CDFs of inter-transaction times for fraud (red) and non-fraud (blue).

One-Step Gaps ($T - T_{-1}$, Figure 6a). Fraud transactions are front-loaded: around 80% of fraud cases occur within just 2000s (under 1.4h) of a previous payment, and the red CDF shoots upward almost vertically in the opening seconds clear evidence of rapid, opportunistic behavior. In contrast,

legitimate activity unfolds more gradually, with its CDF rising steadily over a much broader span. Both curves then plateau almost entirely between 20 000 and 60 000s (5.5–16.7h), hinting perhaps at an overnight or system batch window when virtually no transactions occur. A two-sample Kolmogorov–Smirnov test confirms this separation is not due to chance ($KS = 0.330$, $p \approx 9.2 \times 10^{-129}$).

Two-Step Gaps ($T - T_{-2}$, Figure 6b). By measuring the gap to the transaction two steps prior, the fraud CDF hits 65 % within just 4 000s (1.1h), while legitimate behavior only reaches 45 % at that point. After this threshold, the fraud curve plateaus, showing that most two-step fraud occurs rapidly. In contrast, the non-fraud CDF climbs gradually across a wide range of intervals. This stark “velocity signature” confirms that unusually short two-step gaps are a strong fraud indicator.

Note A Spearman $\rho = 0.533$ ($p \approx 0$) indicates a moderate monotonic link between ΔT and ΔT_2 : knowing the time since the last payment gives some insight into the two-step gap, yet each feature retains unique information. In a tree-based model like XGBoost, this redundancy poses no issue during training, splits are chosen based on whichever feature yields the greatest gain at each node.

5.5 Inter-Transaction Time Analysis

High-value transactions emerge as a decisive fraud signal: the CDF for genuine payments climbs steeply, reaching 80 % by about €500, whereas fraud only hits 40 % at the same threshold.

Beyond €500, 60 % of fraud cases but just 20 % of legitimate transactions reside, pushing the median fraud amount to roughly €700 more than double the €300 midpoint of non-fraud.

This pronounced rightward shift in the fraud curve underscores that larger purchase amounts carry a markedly higher risk and merit strong weighting in any detection model.

This separation was confirmed not to be due to chance by a two-sample Kolmogorov–Smirnov test ($KS = 0.4222$, $p \approx 1.9 \times 10^{-209}$).

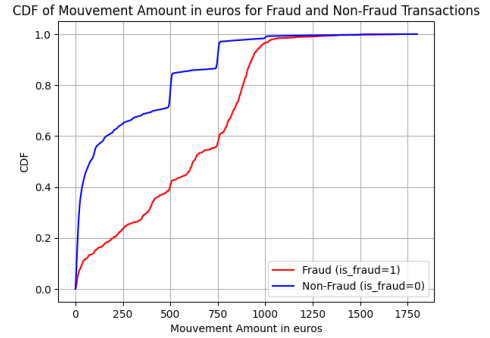


Figure 7: CDF of transaction amounts for fraud (red) and non-fraud (blue).

5.6 Top Currencies: Transaction Share vs. Fraud Rate

The scatter of transaction share versus fraud rate breaks naturally into seven distinct risk volume groupings. Rather than encoding each currency as its own category, we perform K-means clustering ($k = 7$) on their (share, fraud.rate) coordinates and use the resulting cluster label as a compact “risk bucket” feature, capturing these patterns without exploding dimensionality. In the top 15 currencies one can already observe this structure: for example, EUR lies in the high-volume/low-risk bucket, whereas VND appears as a low-volume/high-risk outlier.

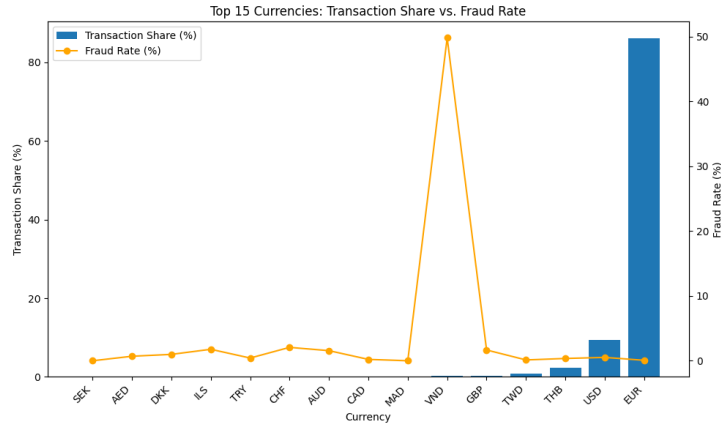


Figure 8: Top 15 currencies by transaction share (bars) and fraud rate (line).

6 Feature Preparation and Transformation

6.1 Chronological Data Splitting

We sorted all transactions by `time_of_transaction` and split chronologically as follows:

- **Train:** up to September 30, 2022 (Jan 1–Sep 30; $n = 391\,631$)
- **Validation:** October 1–November 30, 2022 ($n = 110\,483$)
- **Test:** December 1–December 31, 2022 ($n = 53\,948$)

This forward-in-time split yields fraud rates of

$$\bar{y}_{\text{train}} = 0.002, \quad \bar{y}_{\text{val}} = 0.004, \quad \bar{y}_{\text{test}} = 0.004.$$

After selecting hyperparameters on the January 1–September 30 training set and October 1–November 30 validation set, we retrain the final model on all data from January 1 through November 30 before evaluating performance on the held-out December transactions.

6.2 Currency Fraud–Risk Encoding

On the training set only, we computed for each currency:

$$\text{total_records_pct}, \quad \text{fraud_pct},$$

and then applied K-means clustering ($K = 7$) to these two metrics. Each currency was assigned a `fraud_risk_class` (0–6), with unseen currencies mapped to “unknown.” This compact encoding captures both volume and risk without exploding dimensionality.

6.3 Feature Selection and Encoding

We retained:

- Numerical features: `transaction_amount`, `difference_transaction_time`, `difference_transaction_time2`
- Categorical features: `card_payment_method`, `payment_method`, `country_of_registration`, `fraud_risk_class`

All categorical fields were one-hot encoded and numeric features left at original scale.

7 Model Setup

7.1 Hyperparameter Grid Search

We conducted a chronological grid search using 5-fold `TimeSeriesSplit` on our data, optimizing **mean recall** (the average of fraud and non-fraud recall at a 0.5 threshold).

Importantly, we did not rely solely on the overall mean: for each hyperparameter combination we recorded the mean recall achieved on each fold and selected the configuration that consistently delivered the highest values across all five time-based splits.

Two parameters remained fixed to proven defaults for stability and runtime control:

- `learning_rate` = 0.1 for smooth, incremental tree updates
- `min_child_weight` = 6 to prevent over-splitting under severe class imbalance

We then swept five core knobs:

- `scale_pos_weight` $\in \{1, 100, 200, 411\}$ to rebalance our 1:411 fraud ratio
- `max_depth` $\in \{4, 6, 8\}$ trading off model complexity vs. generalization
- $\gamma \in \{0.0, 0.5, 1.0\}$ as a minimum-gain split penalty
- `subsample`, `colsample_bytree` $\in \{0.6, 0.8, 1.0\}$ for row and feature-level randomness

Each candidate trained up to 200 boosting rounds with early stopping after 10 consecutive non-improving rounds on its validation fold. The optimal setting

$$\text{scale_pos_weight} = 411, \quad \text{max_depth} = 4, \quad \gamma = 0.0, \quad \text{subsample} = 0.8, \quad \text{colsample_bytree} = 0.8$$

not only achieved the highest average mean recall (≈ 0.63) but also topped the per-fold mean recall on each of the five time-based splits, demonstrating strong fraud capture throughout the entire period.

8 Boosting-Round Performance Evaluation

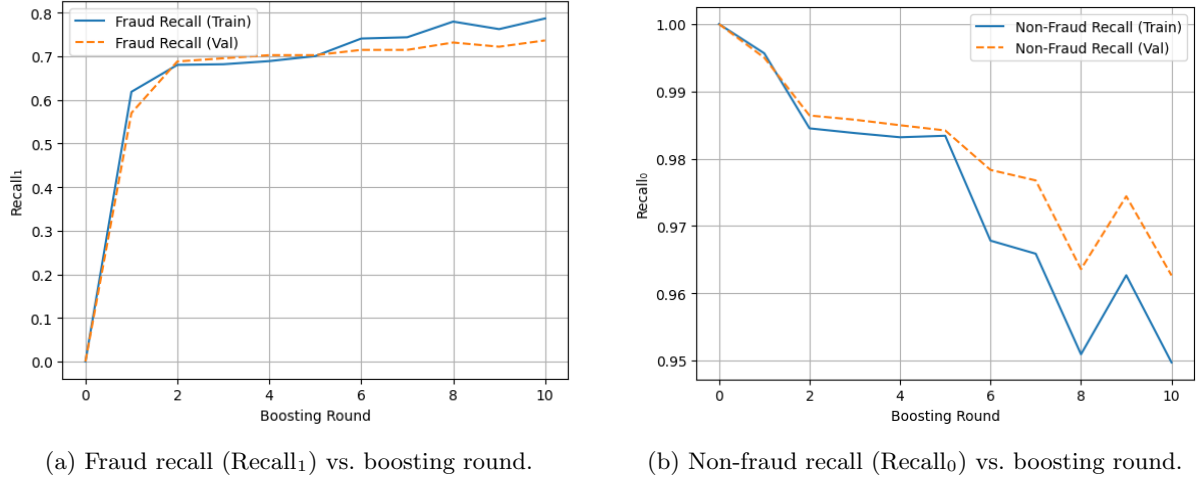


Figure 9: Evolution of fraud and non-fraud recall on train (solid) and validation (dashed) up to the early-stopping round.

Training on our optimal hyperparameters, the model captures most of its fraud-signal in the first few boosting rounds:

- **Fraud Recall (Figure 9a).** Starting at 0% in round 1, recall leaps to $\approx 70\%$ by round 2, to $\approx 75\%$ by round 6, and reaches $\approx 80\%$ by round 9. The validation curve closely tracks within $\pm 1\%$, showing consistent early gains.
- **Non-Fraud Recall (Figure 9b).** Beginning at 100% in round 1, recall dips to $\approx 98.5\%$ by round 3 and to $\approx 98\%$ by round 5, then falls to a minimum of $\approx 95\%$ around round 8 before a slight rebound. Training and validation remain within $\pm 0.5\%$, confirming maintained specificity.

Bottom line: Early-stopping at roughly 8–10 boosting rounds captures nearly all available gain while minimizing training time and overfitting.

8.1 Test-Set Performance

On the held-out December test set (53 948 transactions, 217 frauds), our final XGBoost model achieves an Average Precision (AP) of 0.672 versus a naïve random-ranking baseline of approximately 0.4 % (the fraud prevalence).

Why prevalence equals the PR-curve baseline:

- A “random-ranking” model assigns every transaction an equal chance of being ranked above any other, so it has no information to distinguish fraud from non-fraud.
- In such a ranking, at any recall level R , the expected precision is the overall fraction of frauds in the dataset:

$$\text{Precision}_{\text{random}}(R) = \frac{217}{53\,948} \approx 0.0040.$$

- Hence the Precision–Recall (PR) curve for a random ranker is a horizontal line at 0.0040, and its area the Average Precision is exactly the fraud prevalence.

At the default 0.5 threshold, we flag 191 of 217 frauds (fraud recall 88 %) while misclassifying just 3 790 of 53 731 genuine transactions (false-positive rate 7 %).

In other words, we rank true frauds far above random even a straightforward 0.5 cutoff catches nearly nine-in-ten fraud cases with relatively few false alarms demonstrating that our time-aware tuning delivers strong sensitivity without overly compromising precision on truly unseen data.

8.2 Feature Importance

Our XGBoost model’s top-10 “gain” importances (Figure 10) reveal where it derives its predictive power:

1. **Currency Fraud-Risk Buckets** (`fraud_risk_class_2`, `fraud_risk_class_4`, ...) The K-means-derived clusters of currencies, combining transaction share and historical fraud rate are by far the strongest signals.

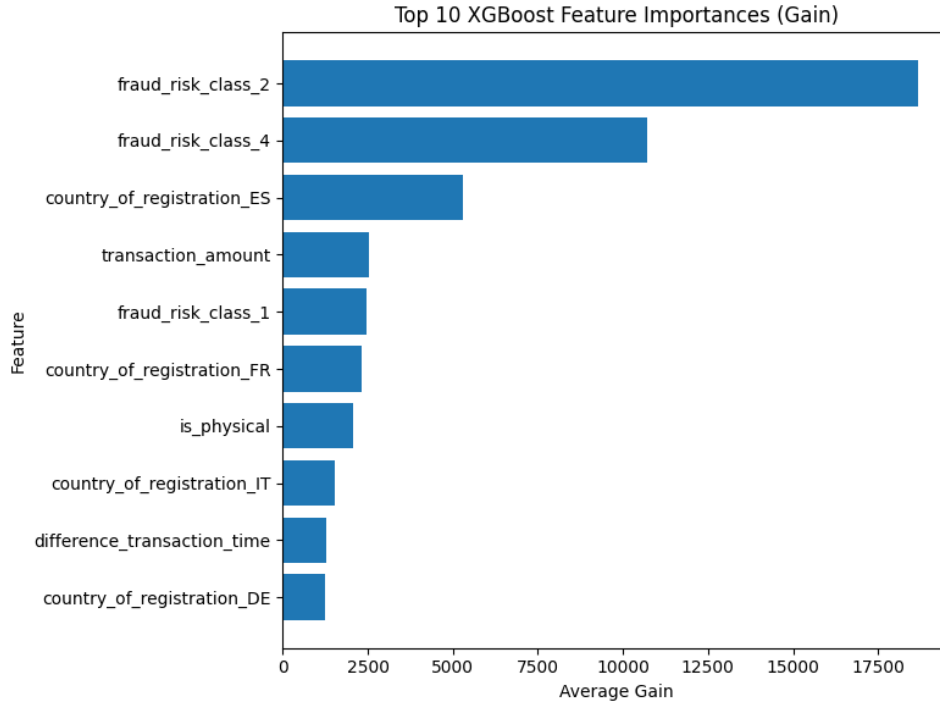


Figure 10: Top 10 XGBoost feature importances (gain).

2. **Issuing Country** (`country_of_registration_ES`, `country_of_registration_FR`, `country_of_registration_DE`) Country flags come next, reflecting regional variations in fraud controls and consumer habits.
3. **Obscure Payment Channels** (`payment_method_unknown`, `payment_method_non_applicable`) Unmapped or “non-applicable” methods show elevated risk, hinting at gaps in our channel profiling.
4. **Transaction Amount** (`transaction_amount`) Larger payments—already identified in EDA significantly increase fraud likelihood, making amount the top numeric driver.
5. **Card-Not-Present Reads** (`card_payment_method_VAD`) Virtual/unattended device (VAD) transactions carry more risk than chip reads, consistent with known vulnerabilities.
6. **Recency of Activity** (`difference_transaction_time`) Rapid, bursty behavior remains a useful secondary predictor, though weaker than our engineered buckets.

High-level engineered features, especially currency-risk clusters and geography account for the bulk of the model’s gains. Raw metrics (amount, time delta) and payment-method indicators add refinements.

9 appendix

Precision and Recall Primer

Definitions

Let

TP = true positives (fraud correctly flagged),
 FP = false positives (legitimate flagged as fraud),
 FN = false negatives (fraud missed).

Then:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

- *Recall* answers: “Of all actual frauds, what fraction did we catch?”
- *Precision* answers: “Of all transactions we flagged, what fraction were truly fraud?”

Intuition in Fraud Detection

- A high recall (e.g. 0.80) means we catch 80 % of fraud cases but says nothing about how many false alarms we raise.

- A high precision (e.g. 0.80) means 80 % of our flagged transactions are real frauds but doesn't tell us how many frauds we might be leaving undetected.

Balancing these is critical: we want to catch as much fraud as possible (high recall) while keeping the review queue manageable (high precision).

Average Precision (AP)

Average Precision condenses the entire Precision–Recall curve into a single score. It is computed as the approximate area under the PR curve (e.g. via `average_precision_score(y_true, y_score)`). Concretely, if you plot precision P vs. recall R as you sweep the classification threshold, then

$$\text{AP} = \sum_{n=1}^N (R_n - R_{n-1}) \times P_n,$$

where each slice $(R_n - R_{n-1})$ is the incremental change in recall and P_n is the precision at that point.

- $\text{AP} \approx 1.0$ Precision stays at 1.0 for every recall level: the model ranks all frauds above all non-frauds, achieving perfect separation (no false positives even at 100 % recall).
- $\text{AP} \approx 0.0$ Precision collapses to (or near) zero as soon as you start flagging. Either you never catch any true fraud ($\text{TP} = 0$) or you immediately swamp your predictions with false alarms.

In practice:

- A high AP indicates your model can steadily raise fraud capture (recall) without precision plunging ideal for rare, high-cost fraud events.
- A low AP warns that even a small increase in catch rate triggers many false positives, making the system impractical to operate.

Thus, Average Precision provides a single, interpretable gauge of your model's ranking quality under extreme class imbalance, directly reflecting its usability in real-world fraud detection.